

Performance & Mobile Constraints

Kursüberblick

1. Intro
2. Mobile UX & Input
-  3. Performance & Mobile Constraints
4. GPS & Location-Based Mechanics
5. Map Integration
6. AR Foundation
7. Game Loop & Systems
8. Polishing + Smartphone Features
9. Prüfung

Heute - Theorie

Die Basics

- FPS und Frames
- Unity Profiler

Performance Killer

- Script Performances & Garbage Collection
- CPU vs GPU: Draw Calls
- Overdraw
- Texture Compression

Heute - Praxis

Fix the lag!

- Optimieren einer ineffizienten Szene

Performance auf Android

Performance - warum jetzt schon?

Performance ist **kein Debugging**. Performance ist **Design**

Am Anfang ist alles Smooth, aber dann:

- mehr Features
- mehr UI
- mehr Content

kleine Dinge, die sich summieren -> und auf einmal laggt!

Und dann zu fixen wird teuer!

Performance - Warum auf Mobile besonders wichtig?

Mobile Hardware ist stark limitiert

- weniger CPU Power
- weniger GPU Power
- weniger RAM
- weniger Speicher
- weniger Kühlung
- kein unbegrenzter Strom

Performance - CPU, GPU

Wir müssen mit weniger **Power** auskommen. Konsequenzen für :

- Meshes
- Texturen
- Shader
- Anzahl an Objekten in Szenen
- komplexe Algorithmen
- Physik

Performance - weniger Kühlung

Kompaktes Gerät ohne Lüfter -> passiv-Kühlung

- heißes Gerät in der Hand zählt nicht als "Hardcore-Gaming!"

Thermal-Throttling:

- wenn das Gerät warm wird, wird die Leistung gedrosselt

-> Die ersten 5 Minuten läuft's, und dann ruckelts!

Performance - Batterie

Strom ist eine begrenzte Ressource!

Mehr benötigte Leistung -> mehr Akkuverbrauch

schlechte Performance = hoher Batterieverbrauch = weniger Spielzeit

Performance - Anforderungen

Anforderungen an Games sind trotzdem hoch!

60 FPS ist Standard!

- mobile sehr nah am Auge
- direkter Input über Touch -> Input Lag wirkt stärker
- kein Display Puffer: V-Sync/FreeSync/G-Sync
- kein "Cheating" mit DLSS/FSR

Kriegen wir aber hin!

FPS

FPS - Der wichtigste Benchmark für alle Games

Möglichst viele und möglichst stabil!

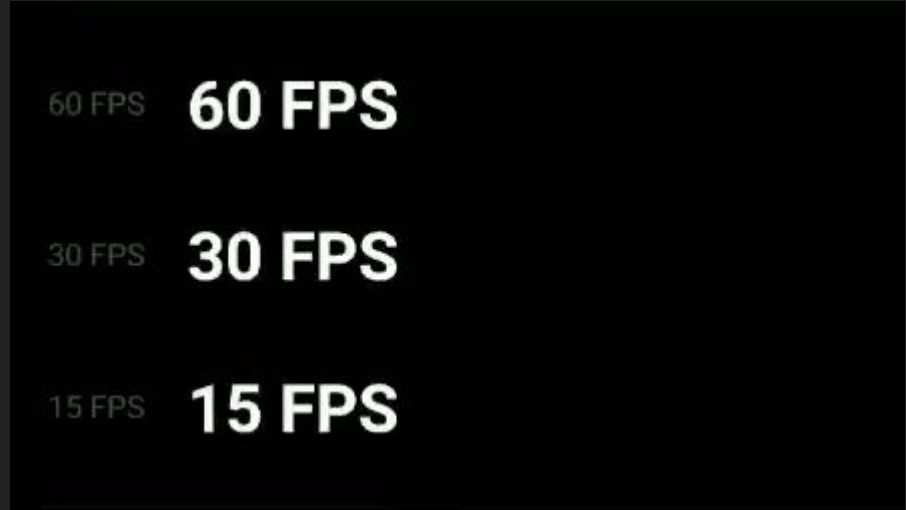
- Warum eigentlich?
- Flüssige Darstellung >>>
- aber auch "Reaktionsfähigkeit"

Wie kriegt man viele Frames?

```
Application.FPS = 60f;
```

```
Application.hasLagSpikes = false;
```

Leider nicht :/

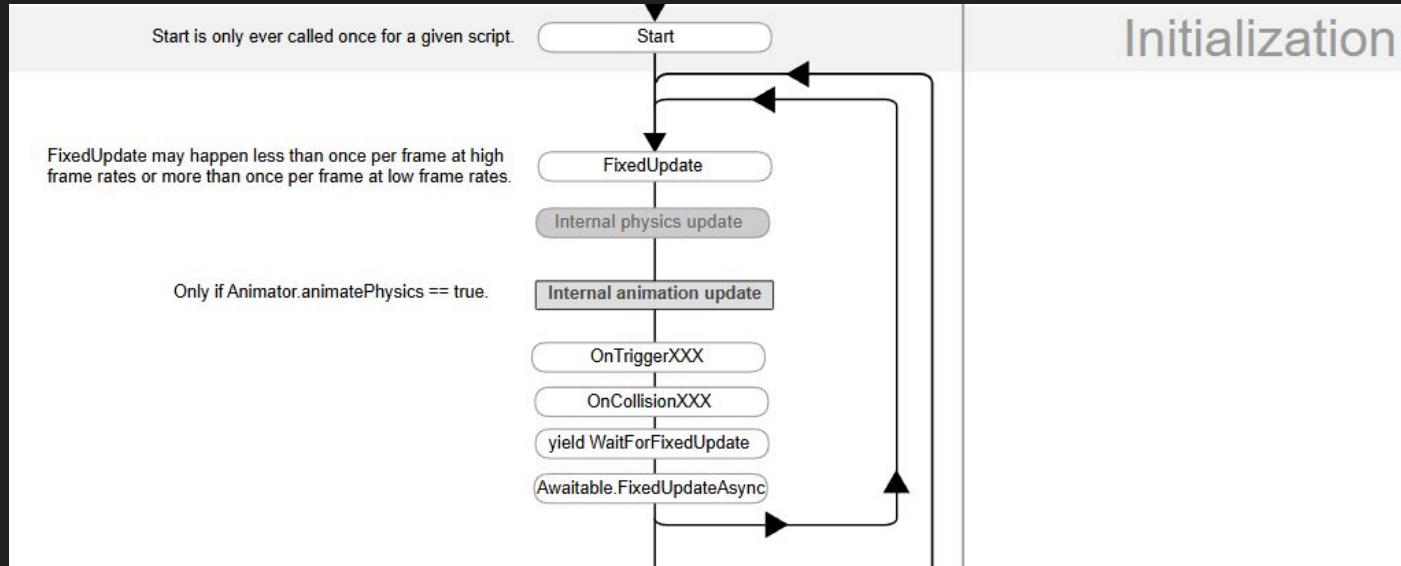


Disclaimer: Gif läuft nur mit 50 FPS, deswegen ist der Unterschied zwischen 60 und 30 FPS so gering

Frame - Was ist das?

Ein “Standbild” in einer Sequenz von Bildern.

Oberflächlich wird “nur” ein neues Bild gerendert.
Im Hintergrund muss aber viel mehr passieren:



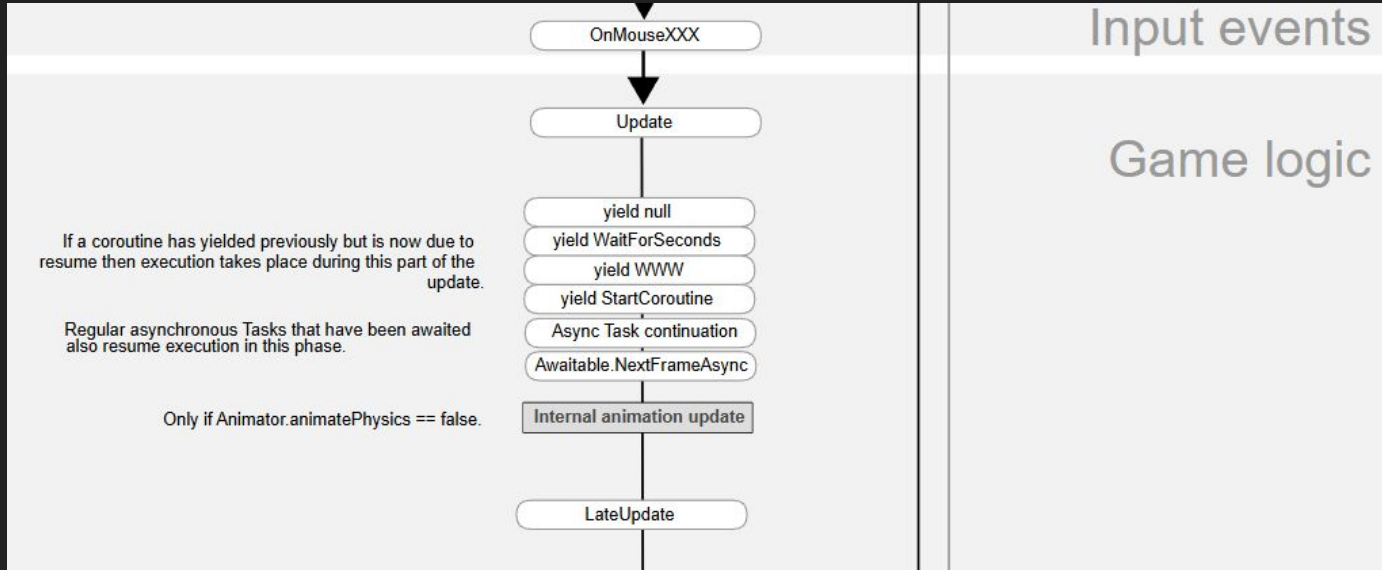
Vorbereitung “neuer” Objekte

Physik Berechnungen

- Trigger
- Collision

Frame - Was ist das? Teil 2

Nach Physik geht's weiter mit Input und Game Logik



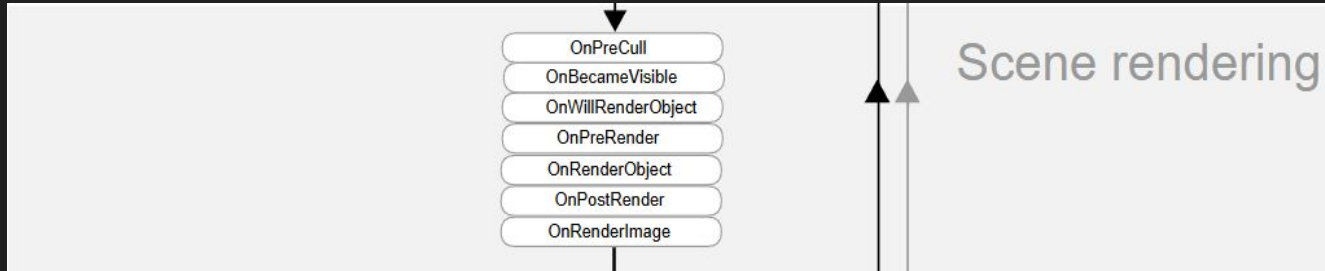
Sämtliche Input Events, z.B.
auch unser Touch

Game Logik,
Coroutinen
Animations Updates

Frame - Was ist das? Teil 3

Anschließend wird gerendert.
Erst die Szene, dann das UI.

Dies ist meistens der “teuerste” Teil im Frame.

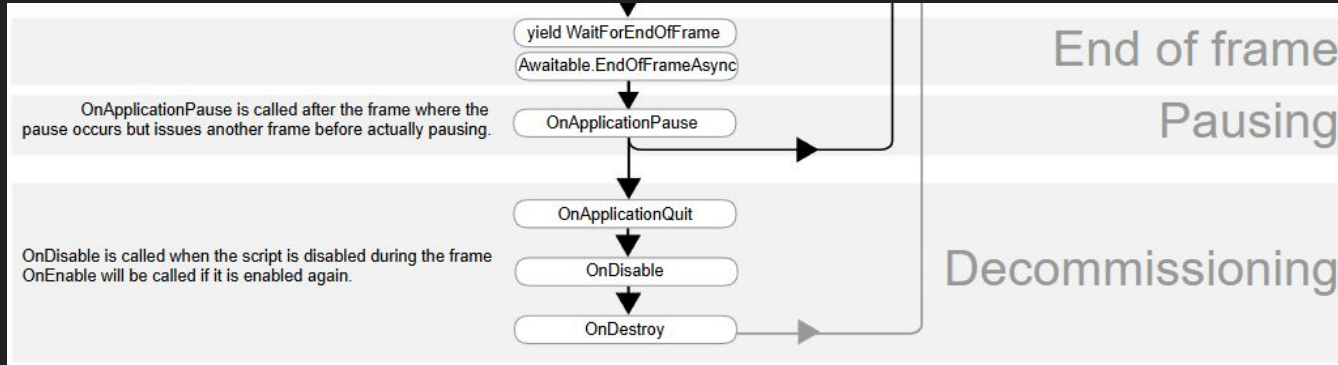


Verschiedenste Rendering
Schritte die hier passieren

Frame - Was ist das? Teil 4

End of Frame und das Aufräumen.

Ja, auch aufräumen muss gehört dazu und kann Kosten verursachen.



Coroutinen die auf das Ende des Frames warten

`OnDisable` - z.B. lösen von Event Listnern, deaktivieren von Child-Objekten und Scripts

`OnDestroy` - Objekte für den Garbage Collector zum wegräumen markieren

Frame Budget

Wir haben ein begrenztes Budget:

Für **60 FPS**: $1\text{s (1000ms)} / 60 \text{ Bilder} = \mathbf{16.6\text{ms pro Frame!}}$

- In diesen 16.6ms müssen alle diese Dinge passieren!
- Alle Systeme teilen sich dieses Budget.
 - Wenn Rendering mehr braucht, darf Physik weniger verbrauchen etc.

Wenn wir **16.5ms** brauchen, ist das nicht schlimm, dann warten wir einfach auf das Ende des Frames.

Wenn wir **16.7ms** brauchen, gibt es einen Framedrop.

Unity Profiler - Live Demo

Was ist das? Was sagt der uns?

Unity Demo: FrameDropDemo

- Analysis -> Profiler

1. Mehr Objekte = mehr Arbeit für alle Systeme
2. Mit Physik wirds noch härter

Script Performance

Game Logic & Script Performance

Jede Zeile Code die ausgeführt wird, kostet Zeit. *Duh!*
Aber nicht jede Zeile ist gleich teuer!

Typische Script-Performance-Killer:

1. Update(), besser als sein Ruf?
2. Caching statt ständiger Lookups
3. Update vs FixedUpdate
4. Instantiate Spam

Update()

Update() wird jedes Frame aufgerufen. (Auch wenn die Funktion leer ist!)

Update ist nicht grundsätzlich schlecht. Das eigentliche Problem ist:

Zu viele Updates mit unnötiger Arbeit!

Daher Update nur nutzen, wenn etwas wirklich in jedem Frame passieren **muss**!

- Visuelle Effekte, Input, Bewegungen

Nicht für:

- Timer, Checks, State Changes

Update() - Beispiel

State Check

- Statt kontinuierlicher Abfrage der Health in jedem Frame
- Health-Check in TakeDamage Funktion
- Wenn kein Schaden bekommt, wird auch kein Code ausgeführt

Erst einmal **minimale Performance** Ersparnis aber bei hunderten von Gegnern skaliert das schnell!

Ständiges prüfen

```
void Update()
{
    if (health <= 0)
        Die();
}
```



Reagieren auf Event

```
public void TakeDamage(int dmg)
{
    health -= dmg;

    if (health <= 0)
        Die();
}
```

Caching statt Lookup

```
player = FindObjectOfType<Player>();
```

Ist ja **nur eine Zeile Code**, kein Problem. Denkste!

Lineare Suche der gesamten Szenen-Hierarchie!

Geht die gesamte Szene Schritt für Schritt durch:

In Update() extrem schlecht!

Aber kann auch Spikes während Start() verursachen.

- Z.B. wenn ein Gegner spawnet.

Scene

GameObject A

Transform

MeshRenderer

Collider

Script

Script

GameObject B

Transform

MeshRenderer

Collider

Script

Script

Caching statt Lookup II

Also stattdessen die Objekte cachen -> zwischenspeichern

Statt jedes Frame neu suchen

```
void Update()
{
    var player = FindObjectOfType<Player>();
    player.Move();
}
```

Nur einmal suchen und speichern

```
Player player;

void Start()
{
    player = FindObjectOfType<Player>();
}
```

Oder noch besser: Direkt im Inspector setzen:

```
[SerializeField]
Player player;
```

Caching statt Lookup III

```
collision.collider.GetComponent<Enemy>().ApplyDamage(30);  
collision.collider.GetComponent<Enemy>().ApplyBurn();  
collision.collider.GetComponent<Enemy>().ApplyStun();
```

Nicht ganz so schlimm, weil es nur **ein** GameObject durchsucht, aber ebenfalls verbesserbar:

Enemy einmal gefunden -> zwischengespeichert. Von dort aufgerufen.

Variable anlegen und **befüllen** ist deutlich günstiger als Listen zu durchsuchen!

```
Enemy enemy = collision.collider.GetComponent<Enemy>();  
enemy.ApplyDamage(30);  
enemy.ApplyBurn();  
enemy.ApplyStun();
```

Update-Varianten - Aufs Timing kommt es an!

Update()

- läuft einmal pro Frame

Gut für:

- Input
- UI
- Animation
- Non-Physics movement

mögliche Fehler:

- Physik-Aufrufe
 - instabile Physik
 - unnötige CPU-Arbeit

FixedUpdate()

- Feste Zeitrates, unabhängig von der FPS
 - Standardmäßig 0.02s

Gut für:

- Rigidbody Physics
- Forces
- Physics Movement

mögliche Fehler:

- schwere Spiellogik während Physik Updates
 - z.B. Pathfinding

Bonus-Killer - Logging

Ja, unsere guten Debug.Logs fressen auch Performance:

- Funktionsaufruf
- String erzeugen
- Parameter auswerten

Noch schlimmer wirds mit aktiviertem Development-Build. Hier kommt noch dazu:

- Logs sammeln
- Stack-Traces erzeugen
- Logs an Konsole senden / Profiler einbinden

Heißt das jetzt, nix mehr zu loggen? Nein! Logs sind wertvoll. Aber im “fertigen Build” sollten sie **sparsam** genutzt werden.

Instantiating-Spam

Unsere Waffe soll Kugeln schießen!

Dafür bauen wir ein **Bullet-Prefab**

Bei jedem Schuss: `Instantiate(prefab);`

Beim Treffer: `Destroy(gameObject);`

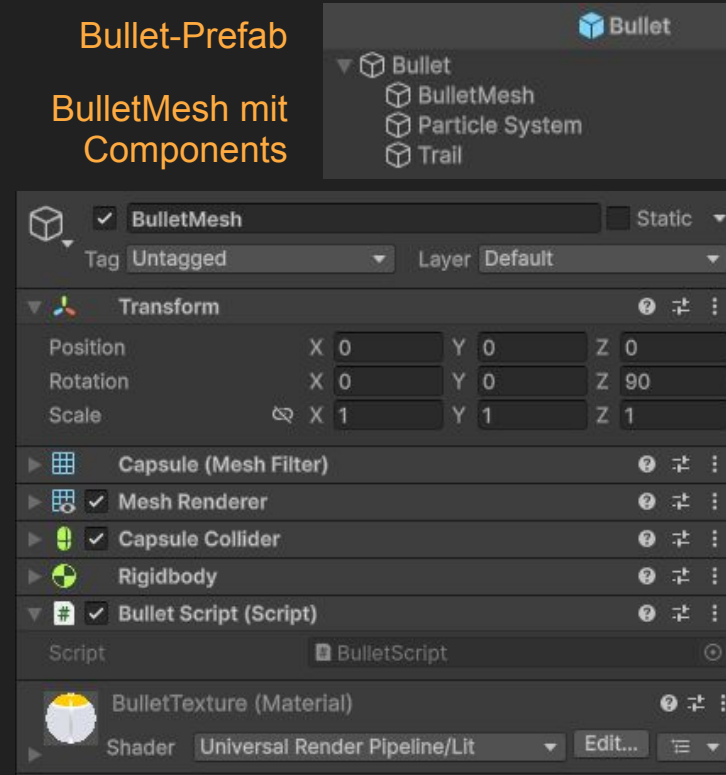
Warum ist das teuer?

- GameObject-Struktur erstellen
- Components initialisieren
- Memory für Transform + Components allokieren
- Awake/OnEnable Lifecycle ausführen
- ggf. Meshes / Renderers / Physics registrieren
- Hierarchie in Szene einhängen

Und bei Destroy wieder rückabwickeln

Bullet-Prefab

BulletMesh mit
Components



Instantiating-Spam-Filter -> Pooling

Wir brauchen die Kugel aber eh beim nächsten Schuss wieder!
Also lieber das **Objekt nur einmal erstellen** und dann **Recyclen!**

Es gibt aber auch Nachteile:

```
bullet.SetActive(true); // reuse  
bullet.SetActive(false); // return to pool
```

- Wir müssen die Kugel jedesmal zurücksetzen:
 - Movement stoppen, Partikel neustarten
- Der Pool muss genügend Kugeln haben, um Schnellfeuer zu schaffen
 - sonst werden die Kugeln zurückgesetzt bevor sie nicht mehr gebraucht werden.
- Die Objekte müssen die ganze Zeit im Speicher gehalten werden, auch wenn sie nicht genutzt werden.
 - Z.B. wenn der Spieler gerade im Stealth Modus ist.

Instantiating-Spam -> Pooling - Wann nutzen?

Durch die Nachteile kann Pooling aber auch schnell **Overkill** werden.

Also nur für Objekte nutzen die häufig gespawnt und zerstört werden.

- Partikel,
- Bullets,
- Gegner

Exkurs - Object Pooling

Wir bauen einen Object-Pool! -> Ab in Unity: ObjectPool Szene

Wir haben:

- **Dispenser**
 - schießt Bullets (*Instantiate()*)
- **Bullets**
 - werden zerstört (*Destroy()*) wenn
 - sie auf ein Objekt treffen
 - oder die maximale Lifetime erreicht ist

Wie können wir das jetzt optimieren?

Exkurs - Object Pooling II

Was brauchen wir alles für den Object Pool?

- Das GameObject, das gepoolt werden soll. -> **GameObject prefab**
- Die Poolgröße, also wie viele Objekte im Pool sein sollen -> **int poolSize**
- Eine Sammlung, in der wir die erstellen Pool Objekte speichern können
 - Sobald wir ein neues Objekt brauchen, soll uns die Sammlung eins geben.
 - irgendeins? Oder ein bestimmtes?
 - Sobald wir das Objekt nicht mehr brauchen, muss es in die Sammlung zurück
 - irgendwo hin?
 - Gibts da nicht was von "Rati.. System.Collections?"
 - **Ja, "Queue<T>"**.

Exkurs - Object Pooling III - Queue

Queue<T>

Stellt eine First-In-First-Out-Auflistung von Objekten dar.

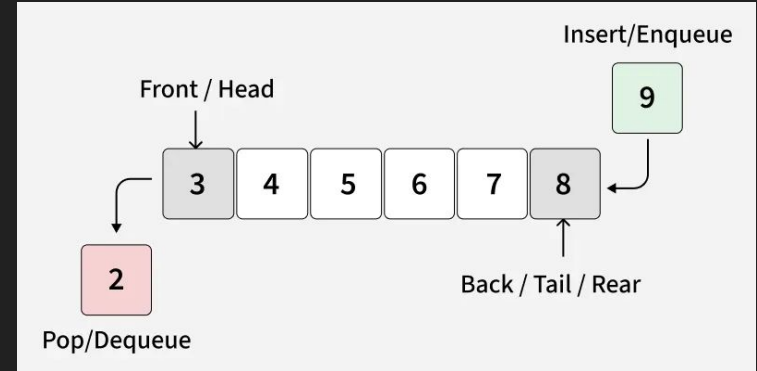
Definition:

```
using System.Collections.Generic;
```

```
private Queue<GameObject> pool = new Queue<GameObject>();
```

Neue Objekte anhängen:

```
pool.Enqueue(obj);
```



Erstes Objekt abrufen

```
GameObject pooledObject = pool.Dequeue();
```

Exkurs - Object Pooling IV

Alles klar. Dann haben wir alles was wir für den Objekt Pooler brauchen:

- PoolObject, Queue, und Queuegröße (Poolgröße)

Und er muss können:

1. **Pool anlegen** (*Initialize()*)
 - a. mit Poolgröße
2. **PooledObject hergeben** (*Dequeue()*)
 - a. Object anmachen
 - b. Wenn keins mehr da ist? Dann eins hinzufügen
3. **PooledObject wieder zurücknehmen** (*Enqueue()*)
 - a. Object wieder ausmachen

Exkurs - Object Pooling V - PooledObject

Der ObjectPooler steht, und unser Dispenser holt sich Objects. Aber:

- Die Bullet wird aktuell zerstört und nicht zurückgegeben!
- Ggf. müssen wir mehr zurücksetzen als nur `.SetActive()`

Wir können die Bullet erweitern um ein *PooledObject* Script. Das soll:

- den Pool kennen
- sich bei Zerstörung/Deaktivierung sich selbst beim Pool zurückgeben
- (optional) mitbekommen, wenn es aktiviert wurde, um Logik auszuführen

Kurze Pause?

Garbage Collection

Garbage Collection - Was ist das?

Beim Programmieren werden ständig Objekte und Variablen erstellt

- z. B. mit `GameObject gameObject = new GameObject();`

Dafür wird Speicher im RAM reserviert

Wenn Objekte nicht mehr gebraucht werden, muss der Speicher wieder freigegeben werden

In Unity übernimmt das der Garbage Collector (GC) automatisch

Garbage Collection - Beispiel

1. Lokale Variable **kiste** -> Ein kleiner **Zettel** auf dem steht "Die Kiste steht in Fach 5"
2. Mit **new OutfitGenerator()** holen wir die große **OutfitGenerator** Kiste und packen sie in Fach 5
3. Ende der Funktion -> **}** erreicht -> **Zettel (kiste)** wird zerknüllt und weggeworfen.

```
void Start()
{
    OutfitGenerator kiste = new OutfitGenerator();
    this.meinShirt = kiste.GetRandomShirt();
}
```

Aber:

Die fette Kiste mit dem **OutfitGenerator** steht noch in Fach 5! Und niemand hat mehr den **Zettel** auf dem steht, wo sie steht!

Sie belegt wertvollen Platz, ist aber völlig nutzlos geworden. Jetzt bräuchte man einen **Müll-Sammler** der sie wegräumen um den Platz für neue Kisten freizumachen!

Garbage Collection

Der Garbage Collector arbeitet nicht durchgehend, sondern in Intervallen.

Das heißt: In bestimmten Momenten läuft er durch und muss den Speicher aufräumen:

1. Alle erreichbaren Objekte finden -> Referenzen verfolgen (auch mehrere Ebenen tief!)
2. Nicht mehr verwendete Objekte identifizieren
3. Den Speicher freigeben und ggf. neu organisieren

Das kostet Zeit und Leistung.

Ihr kennt das: wenn die Putzkolonne im Flur steht, laufen alle vorsichtiger / langsamer.

-> Das sind die sogenannten **GC Spikes**.

Wer mehr ins Detail gehen möchte: <https://youtu.be/0y3erF2tzbl?t=958>

Garbage Collection - Learnings

Der GC ist wichtig und nützlich.

Wenn wir zu sorglos mit dem Speicher umgehen -> Lag Spikes

Deswegen **sparen** mit:

- Allokierungen in Update() vermeiden
- Reuse statt new

```
//Z.B. lieber alte Liste wiederverwenden:
```

```
list.Clear();
```

```
// statt neu anlegen
```

```
list = new List<int>();
```

- Object Pooling

CPU vs GPU

CPU vs GPU

Wer macht was?

CPU - Sequentiell, darin ist die CPU sehr gut. Viele Berechnungen nacheinander!

- die Spiel-Logik,
- die (meiste) Physik
- bereitet die Geometrie vor

GPU - Parallelisiert. Viele Berechnungen nebeneinander!

- Umrechnung der 3D Koordinaten in 2D-Screen Koordinaten
- Rasterisierung, Umwandlung der Vektoren in Pixel (Fragmente)
- Ausrechnen der finalen Farbe für jeden Pixel (Texturen, Beleuchtung)

Fürs Rendering: **CPU** plant, **GPU** führt aus.

Draw Calls

Ein Draw-Call ist ein Befehl der CPU an die GPU:

“Zeichne die Biene dort!”

1. dieses Mesh
2. mit dieser Textur
3. und diesem Shader
4. an dieser Position

Bevor die GPU zeichnen kann, muss die CPU den Draw Call über den Treiber an die Grafikkarte schicken.

- bei 10.000 einzelnen Bienen muss die CPU jedes mal “Zeichne die Biene dort!” rufen
- Das blockiert die CPU!

Draw Calls - Bottleneck

Lösung: GPU Instancing (Stempeln)

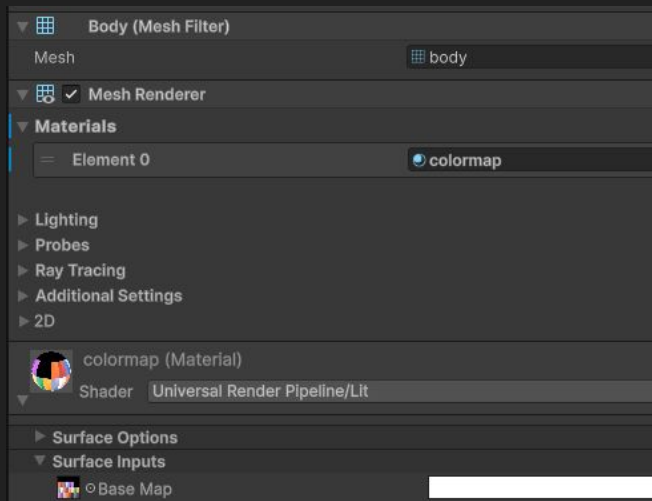
- Die **CPU** fasst möglichst viele **gleiche** Objekte zusammen:
- “Zeichne die Biene an diesen 10.000 Positionen”

Hier ist ein Bienen-Stempel,
der enthält: **Mesh**, **Textur**, **Shader**

- Stempel die Biene mal an diese 10.000 Positionen
 - Ein **einzig**er Draw Call aber 10.000 Bienen.

Und wenn ich nicht nur Bienen habe?

Stempel-Inhalt



Static Batching

Wenn ich mehrere verschiedene Objekte habe, z.B. Steine.

- Alle Steine nutzen die gleiche Textur, aber haben andere Meshes
- Stempel enthält: **Mesh**, **Textur**, **Shader**

Also kein Stempeln möglich? Also wieder 1.000 einzelne Draw Calls?

Nächster Trick: **Static Batching**

- Dazu kombinieren wir alle Steine zu einem großen Mesh!
- Dann ist wieder gleich: **Mesh**, **Textur**, **Shader**
-> 1 Draw Call

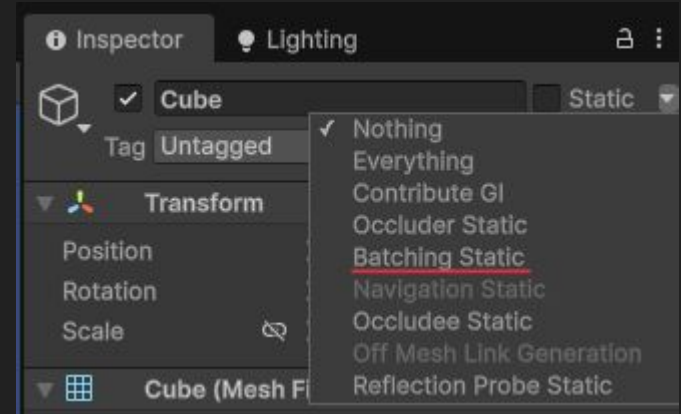
Static Batching

Wie macht man das in Unity?

- Im Inspector, für das Objekt oben rechts auf "Static" -> **Batching Static**

Funktioniert nur wenn:

1. Objekt das **gleiche Material** und **Textur** nutzen
2. Objekte sich **nicht bewegen**



Demo in Unity

Unity's SRP Batchter ist Batching Magie

=> <https://docs.unity3d.com/6000.4/Documentation/Manual/SRPBatcher.html>

- Er **kann** sich über einige der Regeln hinwegsetzen

Wir machen ihn für diese Demo kurz aus, um besseren Vorführeffekt zu bekommen

Texture Atlas

Und wenn Steine nicht alle die gleiche Textur haben?

- Stein A -> Textur **Grau**
- Stein B -> Textur **Schwarz**
- 2 Materials, also leider kein Batching möglich! Oder doch?

Der einzige Unterschied ist die Textur. Also kombinieren wir beide Texturen in einem **Texture Atlas**!

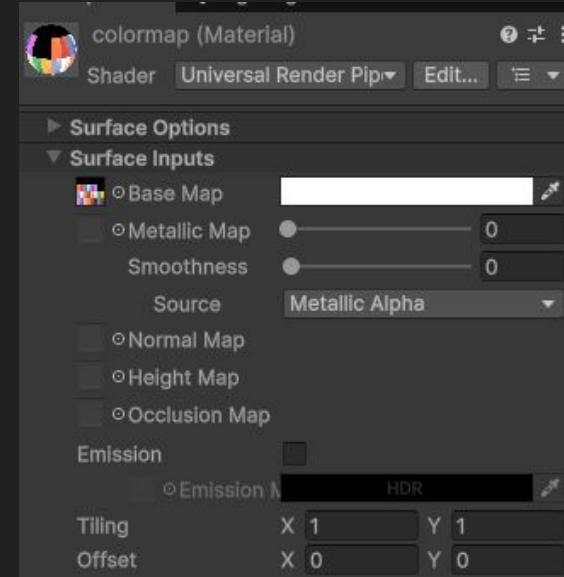
- Jeder Stein benutzt nur einen Teil der Textur, aber:
Dadurch wieder nur **ein Material**!
- Batching wieder möglich -> wieder nur 1 Draw Call!

**Texture-Atlas der
Kenney Models**



Texture Atlas - Limitationen

- Textur Atlas **limitiert die Fläche** die jede einzelne Textur nutzen kann.
- Funktioniert nur bei Shadern/Materials mit **denselben** Eigenschaften
- Muss beim Erstellen des Modells berücksichtigt werden
 - Das UV wird auf entsprechend auf den Atlas gelegt



Tool Empfehlung: MeshBaker im AssetStore

https://assetstore.unity.com/packages/tools/modeling/mesh-baker-5017?srltid=AfmBOoqSZQykmi9gAdCVIWTklK_4wv5EepEnyYIMIGNvmjp-PhVXObr

Alle Materials müssen dieselben Werte haben

Dynamic Batching

Gedacht als “Statisches Batching” für Objekte die sich bewegen.

- CPU macht das statische Batching in jedem Frame neu.
- Harte Limits: Aufgrund der Komplexität nur für Objekte mit **weniger als 300 Vertices!**
- Bei unterschiedlicher Skalierung z.B. (1,1,1) und (2,2,2) bricht es.

Don't use it!

Es ist standardmäßig inzwischen eh aus!

Kurze Pause?

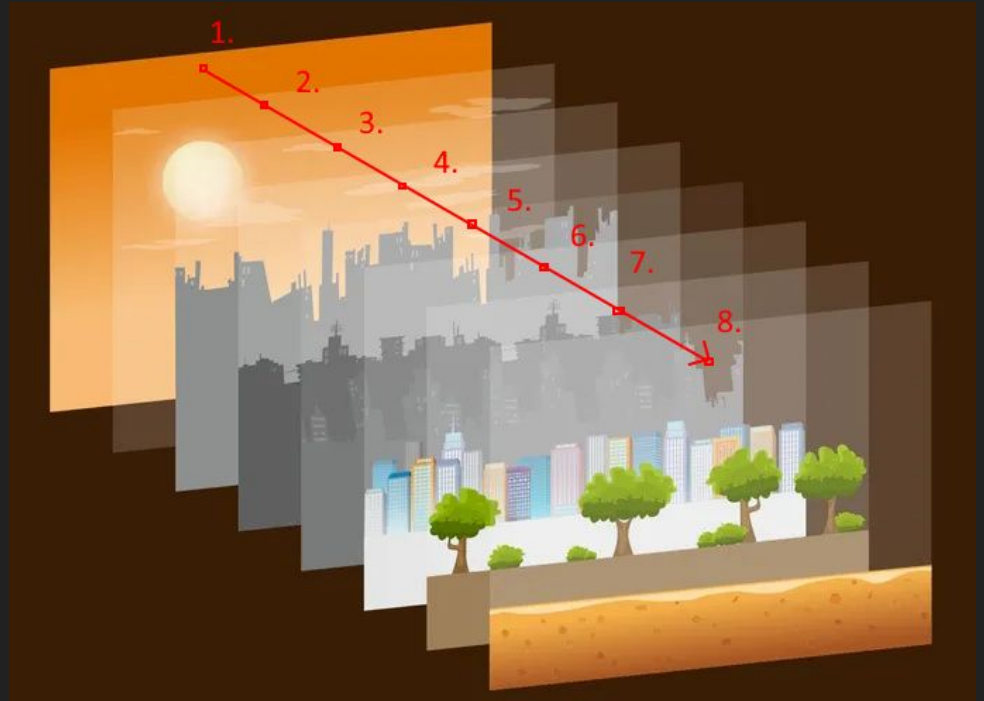
Overdraw

Overdraw

“Pixel werden mehrfach gerendert, weil transparente Objekte übereinander liegen”

1 Pixel - 8 mal rendern

Live Demo in Unity!



Overdraw reduzieren

- Sparsam mit Transparenz:
 - Opaque vor Transparent
 - UI optimieren
- Dinge die komplett verdeckt sind, gar nicht rendern (Layering)
- Sprites - "Tight" Packing
- Sprite Atlas -> Weniger Draw Calls -> Weniger Overdraw

Texture Compression

Texture Compression

Textures sind der größte Memory-Verbraucher.

1024x1024 -> 1.048.576 Pixel

x 4 (RGBA) = 4.194.304 Bytes = **4 MB** unkomprimiert

RTX 5060 = **8GB VRAM**

Mein Pixel => **8GB RAM overall!**

Texture Compression - ETC2

ETC2

- Default für Android (OpenGL ES 3.0)
- Gute Kompression
- Qualität okay
- Blockgröße immer (4x4)

-> Gute Kompression, passable Qualität. Eher als “Fallback”

Texture Compression - ASTC

ASTC

- Moderner Standard!
- deutlich effizienter als ETC2
- bessere Qualität
- variable Blockgrößen (von 4x4 bis 12x12 Pixel)

-> Industriestandard für moderne Mobile-Games

Texture Compression - Vergleich

Uncompressed (RGBA 32bit) > 4MB

ETC2 > 1MB

ASTC 4x4 (High Quality) > 1MB

ASTC 12x12 (High Compression) 115.6 KB

Kurze Pause?

Aufgabe: Fix the scene!

Broken Scene Package herunterladen

(<https://github.com/Tillomaticus/MobileGameProg26Assets/blob/3-Perf/BrokenScene.unitypackage>)

Dort finden sich viele unperformante Dinge.

Versucht alle zu finden und besser zu machen!

Lösung: Fix the Scene

Scene

- Texturen komprimieren
- Background hat overdraw, kann zusammengefasst werden, unnötiger Background (Plane)

Scripts

- Spawner
 - Pooling, Speichern der gespawnten Enemies in Liste statt live suche
- EnemyDecider
 - Statisch machen, immer nur aufrufen, statt instanzieren
- Score
 - Textmesh speichern, string global speichern, Text nur in AddScore updaten
- EnemyScript
 - Unnötiger Material Aufruf verhindert batching
 - Movement in Update statt FixedUpdate
 - ParticleSystem referenzieren statt spawnen
 - unnötiger Update Aufruf mit hit